DEPARTMENT OF CSE(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

DATABASE MANAGEMENT SYSTEMS LABORATORY

Name of the Staff :Dr.B.Venkateswara Rao **Emp Id:11207**
EXERCISE –4: PROGRAMS ON PL/SQL **Date:19/08/2026**

1 a) Experiment: Swap Two Numbers Using PL/SQL

Aim

To write and execute a PL/SQL program to swap the values of two numbers using a temporary variable.

Algorithm

1. Start.
2. Declare three variables: a, b, and temp.
3. Assign values to a and b.
4. Display the values before swapping.
5. Store the value of a in temp.
6. Assign the value of b to a.
7. Assign the value of temp to b.
8. Display the values after swapping.
9. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   a NUMBER := 10;
   b NUMBER := 20;
   temp NUMBER;
BEGIN
   DBMS_OUTPUT.PUT_LINE('Before Swapping:');
   DBMS_OUTPUT.PUT_LINE('A = ' || a);
   DBMS_OUTPUT.PUT_LINE('B = ' || b);

   temp := a;
   a := b;
   b := temp;

   DBMS_OUTPUT.PUT_LINE('After Swapping:');
   DBMS_OUTPUT.PUT_LINE('A = ' || a);
   DBMS_OUTPUT.PUT_LINE('B = ' || b);
END;
/
```

Output

```
Before Swapping:
A = 10
B = 20
After Swapping:
A = 20
```

B = 10

Result

Thus, the PL/SQL program to swap two numbers was executed successfully.

1.b. PL/SQL Program to Find the Largest of Three Numbers

Aim

To write and execute a PL/SQL program to find the largest among three given numbers.

Algorithm

1. Start.
2. Declare three variables a, b, and c.
3. Assign values to the three variables.
4. Compare a with b and c.
5. If a is greater than or equal to both, display a as the largest number.
6. Otherwise, compare b with a and c.
7. If b is greater than or equal to both, display b as the largest number.
8. Otherwise, display c as the largest number.
9. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   a NUMBER := 10;
   b NUMBER := 25;
   c NUMBER := 15;
BEGIN
   IF a >= b AND a >= c THEN
      DBMS_OUTPUT.PUT_LINE('Largest Number = ' || a);
   ELSIF b >= a AND b >= c THEN
      DBMS_OUTPUT.PUT_LINE('Largest Number = ' || b);
   ELSE
      DBMS_OUTPUT.PUT_LINE('Largest Number = ' || c);
   END IF;
END;
/
```

Output

Largest Number = 25

Result

Thus, the PL/SQL program to find the largest of three numbers was executed successfully.

---

2. a. PL/SQL Program to Find the Total and Average of 6 Subjects and Display the Grade

Aim

To write and execute a PL/SQL program to calculate the **total marks**, **average marks**, and **grade** of a student based on marks obtained in six subjects.

Algorithm

1. Start.
2. Declare six variables to store the marks of six subjects.
3. Declare variables for total, average, and grade.
4. Assign marks to all six subjects.
5. Calculate the total marks.

6. Calculate the average by dividing the total by 6.
7. Determine the grade based on the average:
    - Average ≥ 90 → A+
    - Average ≥ 80 → A
    - Average ≥ 70 → B
    - Average ≥ 60 → C
    - Average ≥ 50 → D
    - Average < 50 → F
8. Display the total, average, and grade.
9. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   s1 NUMBER := 85;
   s2 NUMBER := 78;
   s3 NUMBER := 92;
   s4 NUMBER := 88;
   s5 NUMBER := 76;
   s6 NUMBER := 81;

   total NUMBER;
   average NUMBER;
   grade VARCHAR2(2);

BEGIN
   -- Calculate total
   total := s1 + s2 + s3 + s4 + s5 + s6;

   -- Calculate average
   average := total / 6;

   -- Determine grade
   IF average >= 90 THEN
      grade := 'A+';
   ELSIF average >= 80 THEN
      grade := 'A';
   ELSIF average >= 70 THEN
      grade := 'B';
   ELSIF average >= 60 THEN
      grade := 'C';
   ELSIF average >= 50 THEN
      grade := 'D';
   ELSE
      grade := 'F';
   END IF;

   -- Display results
   DBMS_OUTPUT.PUT_LINE('Total Marks = ' || total);
   DBMS_OUTPUT.PUT_LINE('Average Marks = ' || ROUND(average, 2));
   DBMS_OUTPUT.PUT_LINE('Grade = ' || grade);
```

```
END;
/
```

Output

Total Marks = 500
Average Marks = 83.33
Grade = A

Result

Thus, the PL/SQL program to calculate the **total, average, and grade of a student for six subjects** was executed successfully.

---

2. b. PL/SQL Program to Find the Sum of Digits in a Given Number

Aim

To write and execute a PL/SQL program to find the **sum of digits** of a given number.

Algorithm

1. Start.
2. Declare variables num, digit, and sum.
3. Assign a value to num.
4. Initialize sum to 0.
5. Repeat the following steps while num > 0:
    - Find the last digit using MOD(num, 10).
    - Add the digit to sum.
    - Remove the last digit using TRUNC(num / 10).
6. Display the sum of digits.
7. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   num NUMBER := 12345;
   digit NUMBER;
   sum NUMBER := 0;
BEGIN
   WHILE num > 0 LOOP
      digit := MOD(num, 10);
      sum := sum + digit;
      num := TRUNC(num / 10);
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Sum of Digits = ' || sum);
END;
/
```

Output

Sum of Digits = 15

Result

Thus, the PL/SQL program to find the **sum of digits in a given number** was executed successfully.

3. a. PL/SQL Program to Display a Number in Reverse Order

Aim

To write and execute a PL/SQL program to display a given number in reverse order.

Algorithm

1. Start.
2. Declare variables num, digit, and reverse_num.
3. Assign a value to num.
4. Initialize reverse_num to 0.
5. Repeat the following steps while num > 0:
    - Find the last digit using MOD(num, 10).
    - Add the digit to the reversed number.
    - Remove the last digit using TRUNC(num / 10).
6. Display the reversed number.
7. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   num NUMBER := 12345;
   digit NUMBER;
   reverse_num NUMBER := 0;
BEGIN
   WHILE num > 0 LOOP
      digit := MOD(num, 10);
      reverse_num := reverse_num * 10 + digit;
      num := TRUNC(num / 10);
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Reverse Number = ' || reverse_num);
END;
/
```

Output

Reverse Number = 54321

Result

Thus, the PL/SQL program to display a given number in reverse order was executed successfully.

---

3. b. PL/SQL Program to Check Whether a Number is Prime or Not

Aim

To write and execute a PL/SQL program to check whether a given number is **prime or not**.

Algorithm

1. Start.
2. Declare variables num, i, and count.
3. Assign a value to num.
4. Initialize count to 0.
5. Check whether the number is greater than 1.
6. Use a loop from 1 to num.
7. Check whether num is divisible by i.
8. If divisible, increment count.
9. If count is equal to 2, the number is prime; otherwise, it is not prime.
10. Display the result.
11. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   num NUMBER := 17;
   i NUMBER;
   count_num NUMBER := 0;
BEGIN
   FOR i IN 1..num LOOP
      IF MOD(num, i) = 0 THEN
         count_num := count_num + 1;
      END IF;
   END LOOP;

   IF count_num = 2 THEN
      DBMS_OUTPUT.PUT_LINE(num || ' is a Prime Number');
   ELSE
      DBMS_OUTPUT.PUT_LINE(num || ' is not a Prime Number');
   END IF;
END;
/
```

Output

17 is a Prime Number

Result

Thus, the PL/SQL program to check whether a given number is **prime or not** was executed successfully.

---

4. a. PL/SQL Program to Find the Factorial of a Given Number

Aim

To write and execute a PL/SQL program to find the **factorial of a given number**.

Algorithm

1. Start.
2. Declare variables num, i, and fact.
3. Assign a value to num.
4. Initialize fact to 1.
5. Use a loop from 1 to num.
6. Multiply fact by each number in the loop.
7. Display the factorial value.
8. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   num NUMBER := 5;
   fact NUMBER := 1;
BEGIN
   FOR i IN 1..num LOOP
      fact := fact * i;
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Factorial of ' || num || ' = ' || fact);
END;
```

/

Output

Factorial of 5 = 120

Result

Thus, the PL/SQL program to find the **factorial of a given number** was executed successfully.

---

b. Write a PL/SQL code block to calculate the area of a circle for a value of radius varying from 3 to 7. Store the radius and the corresponding values of calculated area in an empty table named areas, consisting of two columns radius and area.

Aim

To write and execute a PL/SQL program to calculate the **area of a circle** for radius values from **3 to 7** and store the radius and corresponding area values in a table named AREAS.

Algorithm

1. Start.
2. Create an empty table named AREAS with columns RADIUS and AREA.
3. Declare a variable to store the calculated area.
4. Use a FOR loop to vary the radius from 3 to 7.
5. Calculate the area using the formula:
   Area = π × radius × radius
6. Insert the radius and calculated area into the AREAS table.
7. Display the stored values.
8. Stop.

Step 1: Create the Table

```
CREATE TABLE areas (
   radius NUMBER,
   area NUMBER(10,2)
);
```

Step 2: PL/SQL Program

```
SET SERVEROUTPUT ON;

DECLARE
   v_area NUMBER(10,2);
BEGIN
   FOR r IN 3..7 LOOP

      v_area := 3.14159 * r * r;

      INSERT INTO areas(radius, area)
      VALUES (r, v_area);

   END LOOP;

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Area values inserted successfully.');

END;
/
```

Step 3: Display the Table Records

```
SELECT * FROM areas;
```

Output

Radius	Area
3	28.27
4	50.27
5	78.54
6	113.10
7	153.94

Result

Thus, the PL/SQL code block to calculate the area of a circle for radius values from **3 to 7** and store the results in the AREAS table was executed successfully.